

COMP3330 HWA 2

Jordan Haigh c3256730

Evan Gresham c3196094

Sebastian Wallman c3162759

Q1 CNNs for classification

Part i) CNN from scratch

For this assignment, it was decided to use the 360 Fruits Dataset found from Kaggle.com (<https://www.kaggle.com/moltean/fruits>). This dataset contained 64 different classes of fruits that were rotated slowly and captured and captured by a webcam. For the CNN that was built, it was decided to reduce this to 10 classes to minimise time spent running a session.

Each image was 100 pixels long by 100 pixels wide and were categorised into 10 groups (Apple, Avocado, Banana, Blueberry, Lemon, Orange, Pineapple, Pomegranate, Raspberry, Strawberry).

Data was already provided for the Training and Validation sets, a test data set had to be created for testing the trained CNN after completion. Training and validation datasets were split into sub directories, each containing the images of a certain fruit.

Using the image data generator, variations to the training data were created. This included changing arguments such as:

`'resize = 1/255'`

Used to scale the pixel values, which range from 0 – 255, to between 0 and 1

`'shear_range = 0.2'`

Applies a shear transformation to each image, generating more training data.
i.e. creating images of fruits which are slightly distorted.

`'zoom_range = 0.2'`

Creates new images by zooming in on the image at different points, thus generating images of fruit at different sizes

`'horizontal_flip = true'`

Flips every image horizontally to double the number of images.

The images were then loaded into the data generator using the `flow_from_directory()` function.

This function resizes the images and then converts the images to tensors (matrices) and gives them a label based on the sub directory they are found in. The labels are then *one-hot*ted. With 10 fruits, there must be 10 outputs to our CNN, the highest output for our CNN will be the network's prediction.

The *flow_from_directory()* function also handles converting the images into the desired size. This dataset has all images at the same size so there is no need to alter image sizes, they will remain at 100 pixels x 100 pixels. The function also splits the data into batches for training.

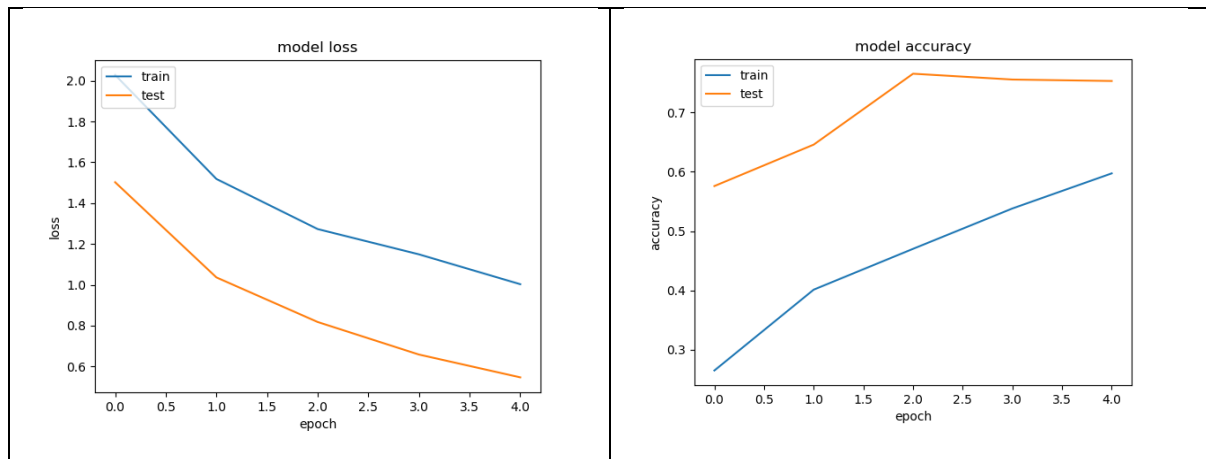
This process was repeated to create the validation dataset.

Using the data generators, the model could be fit using *fit_generator()* method. This uses the data generator to create new images in real time such that the generated images don't need to be stored. The model is then trained with this data.

The *steps-per-epoch* argument was set to be (*half the number of samples*)/ *batch size*. This means that the model would only be trained on half the images per epoch. This was implemented to improve the visualisation of the accuracy and loss graphs as there would be twice the number of plotted points (See Figures 1 and 2). The images were also shuffled so that the model wouldn't be trained on the same images each epoch.

As for the layers of the CNN, it was advised to view example code provided in the lab content of this course. This formed a foundation for the implementation of our CNN. The only difference is the final layer which needed to be changed to have 10 outputs.

The first session could run for only 5 epochs, each epoch taking about 40 seconds, a total of about 4 minutes. It achieved an accuracy of 59.40%.



Figures 1 and 2. Improvements in model loss and accuracy across the five epochs in this session, achieving an accuracy of 59.4%. Though still not an acceptable result.

The CNN struggled with was classifying fruits that are round. For example, apples, pomegranates and oranges are all similar in shape, and were all being classified as apples (See Figure 3).

The exception to the round fruit issue was blueberries. It is hypothesized that the colour difference between blueberries and the other fruits is sufficient to differentiate them, since the images are split into RGB matrices the blueberries will have a very high blue value, whereas most other fruit are much higher in red and green values, hence it is much harder to differentiate between orange and red. It is noteworthy that raspberries were classified with a 99% accuracy. This is due to their unique colour and shape.

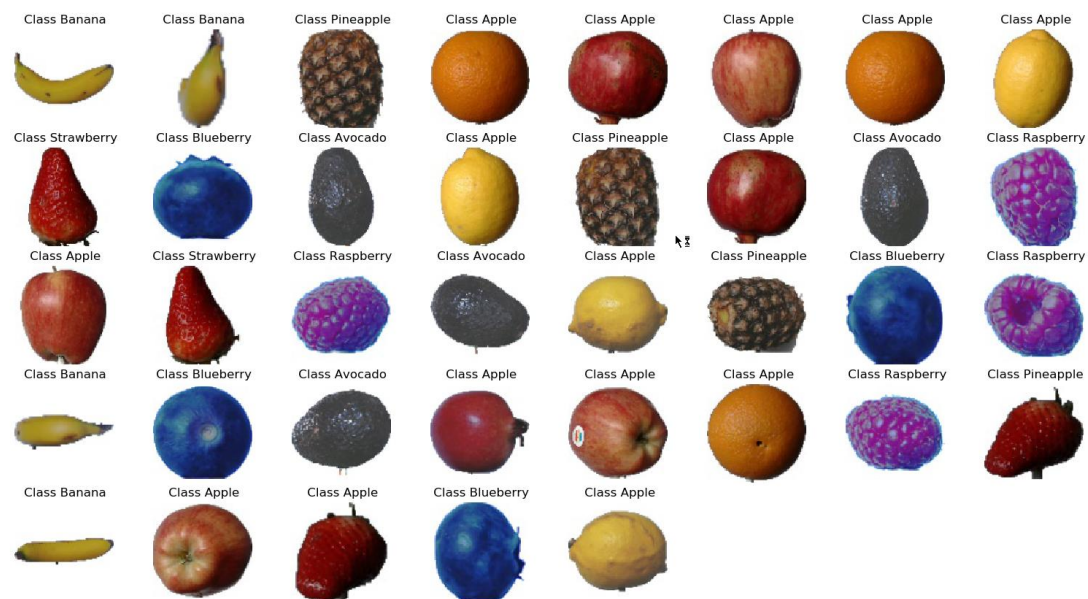


Figure 3: Test Dataset of Fruits run through the CNN. Fruits such as Oranges and Pomegranates were classified as apples mistakenly.

To improve on this accuracy, the number of epochs were increased to 20, taking approximately 15 minutes to complete. This session performed greater than the predecessor, achieving an accuracy of 96.94% (See Figure 4).

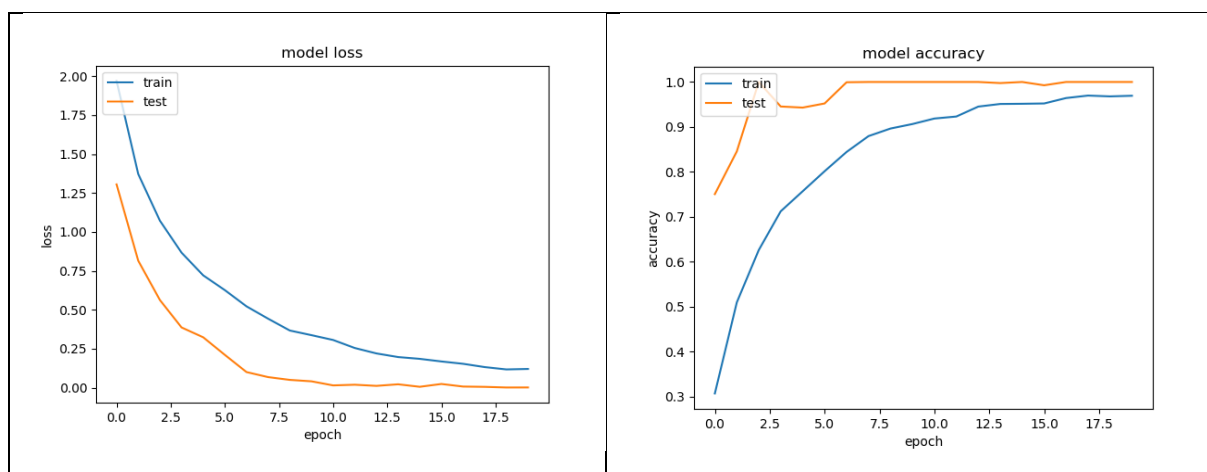


Figure 4. Greater improvements taken to minimise the loss and maximise the accuracy as best as possible, achieving an accuracy of 96.94%.

This session could classify many of the fruits which the previous model struggled with. The exception being differentiating between pomegranates and apples. This was not surprising as both fruit are rather similar, in shape and in colour (See Figure 5).



Figure 5: Misclassification of various pomegranates as apples.

The attempt to increase the number of epochs had very little effect shown by the accuracy function. Increasing accuracy from this stage proved difficult.

The selection of an appropriate batch size was crucial to the success of the model, as a batch size which is too high would slow the training down and a batch size too small would increase the chance of overfitting the model to the images in each batch. As such, finding an optimal batch size increases the accuracy of the model. The original batch size of 16 was found to be an equal contender for optimal size.

Part ii) Pre-trained CNN

This section compares training a handmade model from the beginning as opposed to training a model using a pretrained model. It was decided to use VGG16 as the pretrained base for this section. VGG16 is a convolutional neural network that was submitted in 2014 to the Imagenet Large Scale Visual Recognition Challenge (ILSVRC), it was constructed to be an improved version of the network detailed by K.Simony (Very Deep Convolutional Networks for Large-Scale image recognition).

All the data generation methods were the same as the previous model. On top of the pretrained VGG16 layers 2 dense layers with 512 neurons in each and the 10 output neurons required for classification were added.

For our first attempt using VGG16, it was decided to use 5 epochs and immediately became evident that each epoch in this model would take much longer than our attempt at a CNN. It took about 9 minutes per epoch, a substantial increase to the 40 seconds for each epoch in the CNN that was created by the team.

Despite how long it took, the pretrained CNN achieved a much greater accuracy than the model the team had created. By the end of the first epoch the model achieved an accuracy of 80% and by the end of the fifth epoch the model achieved a perfect accuracy of 100% (See Figure 6).

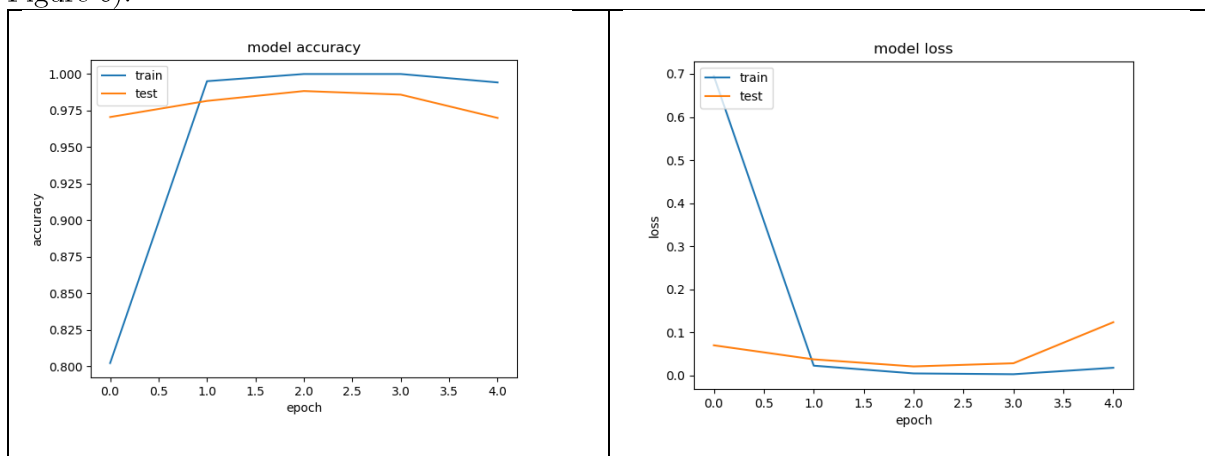


Figure 6: Outstanding performance by VGG16 completing the classification within three epochs.

This model achieved 100% by the third epoch so training it for five epochs wasn't necessary. It is evident that the accuracy decreases after the fourth epoch (represented as the third epoch on the X axis of the Accuracy graph). This is due to the model being near perfect by the end of the third epoch; thus, further training means that the model will be over fitted to the images in the training set.

The VGG16 model had no issue with differentiating between apples and pomegranates, which all previous models failed to successfully classify.

Comparison

The CNN model that was created by the team was much quicker to train than the pretrained model. The total training time of our CNN took approximately 15 minutes, compared to the time it took to train the pretrained model which was about 40 minutes. This was mainly due to the size of the CNNs. our CNN model consisted of 3 convolution and pooling layers, a 64-neuron dense layer and finally the 10-neuron output layer.

The pretrained model on the other hand had many more layers and parameters to train. Figure 7 contains a comparison between the layouts of our CNN model compared to VGG16.

Layer (type)	Output Shape	Param #	Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 98, 98, 32)	896	input_1 (InputLayer)	(None, 100, 100, 3)	0
activation_1 (Activation)	(None, 98, 98, 32)	0	block1_conv1 (Conv2D)	(None, 100, 100, 64)	1792
max_pooling2d_1 (MaxPooling2D)	(None, 49, 49, 32)	0	block1_conv2 (Conv2D)	(None, 100, 100, 64)	36928
conv2d_2 (Conv2D)	(None, 47, 47, 32)	9248	block1_pool (MaxPooling2D)	(None, 50, 50, 64)	0
activation_2 (Activation)	(None, 47, 47, 32)	0	block2_conv1 (Conv2D)	(None, 50, 50, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 23, 23, 32)	0	block2_conv2 (Conv2D)	(None, 50, 50, 128)	147584
conv2d_3 (Conv2D)	(None, 21, 21, 64)	18496	block2_pool (MaxPooling2D)	(None, 25, 25, 128)	0
activation_3 (Activation)	(None, 21, 21, 64)	0	block3_conv1 (Conv2D)	(None, 25, 25, 256)	295168
max_pooling2d_3 (MaxPooling2D)	(None, 10, 10, 64)	0	block3_conv2 (Conv2D)	(None, 25, 25, 256)	590080
flatten_1 (Flatten)	(None, 6400)	0	block3_conv3 (Conv2D)	(None, 25, 25, 256)	590080
dense_1 (Dense)	(None, 64)	409664	block3_pool (MaxPooling2D)	(None, 12, 12, 256)	0
activation_4 (Activation)	(None, 64)	0	block4_conv1 (Conv2D)	(None, 12, 12, 512)	1180160
dropout_1 (Dropout)	(None, 64)	0	block4_conv2 (Conv2D)	(None, 12, 12, 512)	2359808
dense_2 (Dense)	(None, 10)	650	block4_conv3 (Conv2D)	(None, 12, 12, 512)	2359808
activation_5 (Activation)	(None, 10)	0	block4_pool (MaxPooling2D)	(None, 6, 6, 512)	0
Total params: 438,954			block5_conv1 (Conv2D)	(None, 6, 6, 512)	2359808
Trainable params: 438,954			block5_conv2 (Conv2D)	(None, 6, 6, 512)	2359808
Non-trainable params: 0			block5_conv3 (Conv2D)	(None, 6, 6, 512)	2359808
			block5_pool (MaxPooling2D)	(None, 3, 3, 512)	0
			flatten_1 (Flatten)	(None, 4608)	0
			FC1 (Dense)	(None, 512)	2359808
			dropout_1 (Dropout)	(None, 512)	0
			FC2 (Dense)	(None, 512)	262656
			dense_1 (Dense)	(None, 10)	5130
			Total params: 17,342,282		
			Trainable params: 15,606,794		
			Non-trainable params: 1,735,488		

Figure 7: Comparison between the layouts of our CNN model (Left) and the VGG16 model(Right)

The pretrained model had over 15 million trainable parameters whereas the from scratch model only had less than 500,000. Therefore, there were over 30 times more parameters to train in the pretrained model then there were in the from scratch model. This would take much longer to train.

However, the accuracy of the models is where the benefits of using a pretrained model became apparent. The pretrained model achieved an accuracy of 100% within two epochs, while the from scratch model only achieved a 96.94% accuracy after 20 epochs and wasn't going to get much better than 97% without an absurd amount of training.

Conclusion and Final Thoughts

Using a pretrained model seems to be advantages in almost all cases, while you could train a smaller model from scratch faster, for any reasonably difficult image classification task, the pretrained model is going to be far superior in terms of accuracy, which is far more important than the training time.

The fruit 360 dataset was a rather simple classification task with no difference in size of objects nor were there backgrounds or other objects to create noise in the images. This meant that the CNN required to get a good accuracy didn't need to be that deep, which was why our model was able to achieve an accuracy of 96.94% within 15 minutes.

The pretrained model was still far superior despite its size, thus even with very simple tasks using a pretrained model is very beneficial.

Q2 CNNs for Localisation

Labelling data

For this section, a dataset of personal key images needed to be created. For this the team took over 200 photos of various keys in different locations, capturing a vast range of locations and distances from the keys. These images were then processed through python scripts generated by the team to flip the images and rotate them by 180 degrees to increase the size of the training data.

To label the images it was decided to create a program to label the images. This program was written in Processing, allowing the user to draw bounding boxes around keys faster than other labelling tools.

This program loads in an array of images then displays all the images (one at a time). The user clicks once for the top left and again for the bottom right of the image. Once the user is happy with the bounding box, a left click anywhere on the screen will move to the next image. To redo the bounding box, the user must use right click and the box will be deleted.

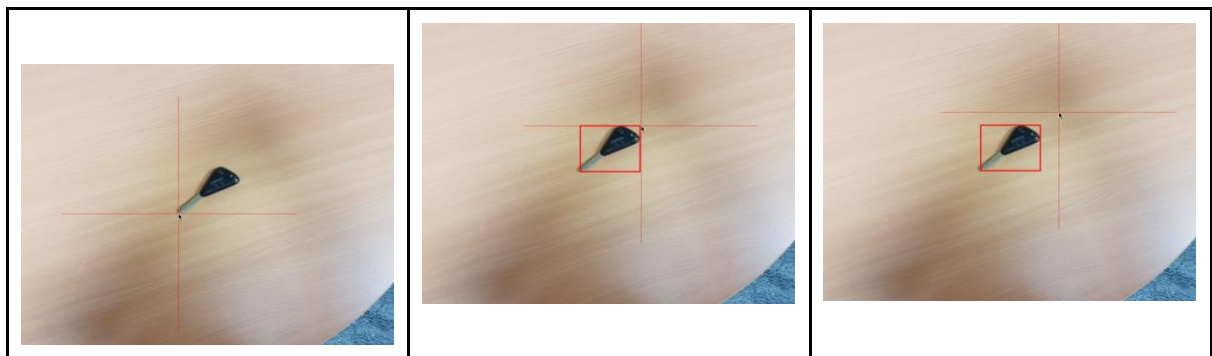


Figure 8 - Our bounding box program

The program then outputs a xml file with all the details of the bounding boxes for all the images.

The reason to create our own program was to increase the number of pictures that could be labelled per minute (LPM). Most of the free programs which that were tried required the user to manually save each label then move to the next image. This is all automated in the program created by the group.

The program 'LabelImg' (<https://github.com/tzutalin/labelImg>) was investigated as a possible label making tool. In order to create a label, the following steps need to be taken:

1. Click to create bounding box
2. Click and drag the bounding box until happy with the size and position of the box
3. Enter the name of the label
4. Save the label to an XML file
5. Click to move to the next image

While this may not seem time consuming, when the user is completing the same process for every image (200+), it affects the overall time of completing this task, with a LPM of approximately 8.

The label making tool that the team created boasts an LPM of over 15. (1 image every 4 seconds).

After this process, the bounding boxes XML files need to be converted to CSV, which is discussed in detail below.

MobileNet

For this assignment it was decided to use a pretrained CNN named *MobileNet*. MobileNet uses depth-wise separable convolutions to reduce the number of parameters to in the CNN, which will drastically reduce the training time and required computations. The model includes convolution layers, depth wise convolutions and pointwise convolutions

A single Depthwise-Separable convolution is a 2-convolution sequence which first applies Depthwise convolution followed immediately by pointwise convolution.

Depthwise convolution.

Depthwise convolution uses filters which only have a depth of 1, which means that each filter can only detect features in a certain layer of the input matrix. This would normally mean that they would be considerably less effective as normal convolutions as they cannot find features which span multiple layers. Therefore, they are always immediately followed by pointwise convolution.

Pointwise convolution

Pointwise convolution is the exact opposite as depth wise, instead of having filters with only depth of one pointwise only has a width and height of 1 but the maximum depth. This is applied after Depthwise convolution, which looks for features in a single layer, the pointwise convolution then combines the features found in all.

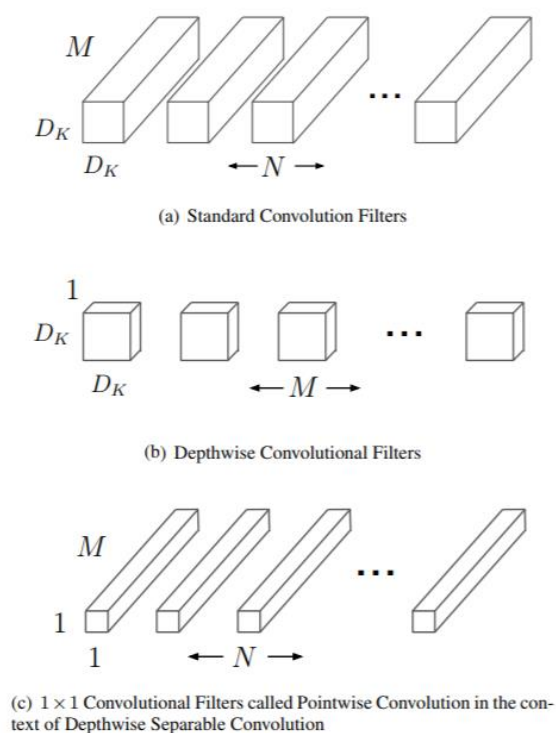


Figure 9

Depthwise Separable Convolutional vs normal convolution

While Depthwise separable convolutions are still not as accurate as full convolutions, the number of parameters to train is drastically reduced.

Table 4. Depthwise Separable vs Full Convolution MobileNet

Model	ImageNet Accuracy	Million Mult-Adds	Million Parameters
Conv MobileNet	71.7%	4866	29.3
MobileNet	70.6%	569	4.2

Figure 10 – Comparing Full Convolutions to Depthwise Separable Convolutions

The full convolution equivalent of MobileNet has an accuracy of 71.7% versus an accuracy of 70.6% for the Depthwise separable convolution, this is a relatively small reduction in accuracy for the extreme reduction in parameters. Using DSC (Depthwise separable convolution) reduced the number of parameters by 85%.

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
		$14 \times 14 \times 512$
Conv dw / s2	$3 \times 3 \times 512$ dw	$14 \times 14 \times 512$
Conv / s1	$1 \times 1 \times 512 \times 1024$	$7 \times 7 \times 512$
Conv dw / s2	$3 \times 3 \times 1024$ dw	$7 \times 7 \times 1024$
Conv / s1	$1 \times 1 \times 1024 \times 1024$	$7 \times 7 \times 1024$
Avg Pool / s1	Pool 7×7	$7 \times 7 \times 1024$
FC / s1	1024×1000	$1 \times 1 \times 1024$
Softmax / s1	Classifier	$1 \times 1 \times 1000$

Figure 11: MobileNet architecture

Since our team is going to use a CPU to train our CNN, having a fast to train CNN is essential. That is why MobileNet was chosen.

Setting up the environment

The setup and installation process for TensorFlow on a Windows machine was confusing, requiring specific versions (recent versions were using deprecated features, sometimes throwing errors) of programs such as *Cuda*, *Cudnn*, *TensorFlow*, *Anaconda* and *Python*. After this setup and installation, it still may not work on a Windows machine.

After encountering numerous issues and extensive lengths of troubleshooting, it was decided to use a Linux based Operating System instead.

It was decided to research and use the TensorFlow Object Detection API and model it for this assignment. Before diving into machine learning, the XML labels created from our tool had to be converted to a format to be fed to the TensorFlow Object Detection API.

XML To CSV

A python script was created that would import an XML file and convert it to a CSV Format. This was so that the important information could be extracted from our label making tool (ImageName, NameOfLabel, Width, Height, StartX, StartY, EndX, EndY).

It was hopeful that the Pandas library in Python would support a conversion from XML to Data-frame though this does not exist. This is due to XML files having a tree structure.

The python script that was created by the team replicated this functionality of converting an XML File to DataFrame. The script was heavily tailored to read attributes that are meant to belong to the XML files that are being used for this project.

TF Record Files

The TensorFlow Object Detection API does not use CSV files as the input, meaning that the CSV Files had to be converted to the appropriate format. For assistance in this task, it was suggested to look to a useful python script from a user on GitHub:

(https://github.com/datitran/raccoon_dataset/blob/master/generate_tfrecord.py)

Training and Evaluation

To use the TensorFlow Object Detection API, the team needed to determine which pretrained model to use with our training. It was decided to use the *MobileNet_v1_coco* model due to the speed at which it can classify keys. After deciding on a model, a matching configuration file needed to be downloaded. This configuration file specified the location of the TensorFlow records and output documents generated by TensorFlow.

The model was trained for a few hours with a variety of training images. The progress could be checked using a pre-prepared Jupyter file and testing images never seen before.

Initially the network was trained on 650 images of varying difficulty for 3 hours, however it performed poorly. It was unable to classify images that contained only a key on a solid background and misclassified keys in images with varying amounts of noise. It is believed that the short training time combined with the high portion of difficult photos was causing the network to be unable to classify a key.

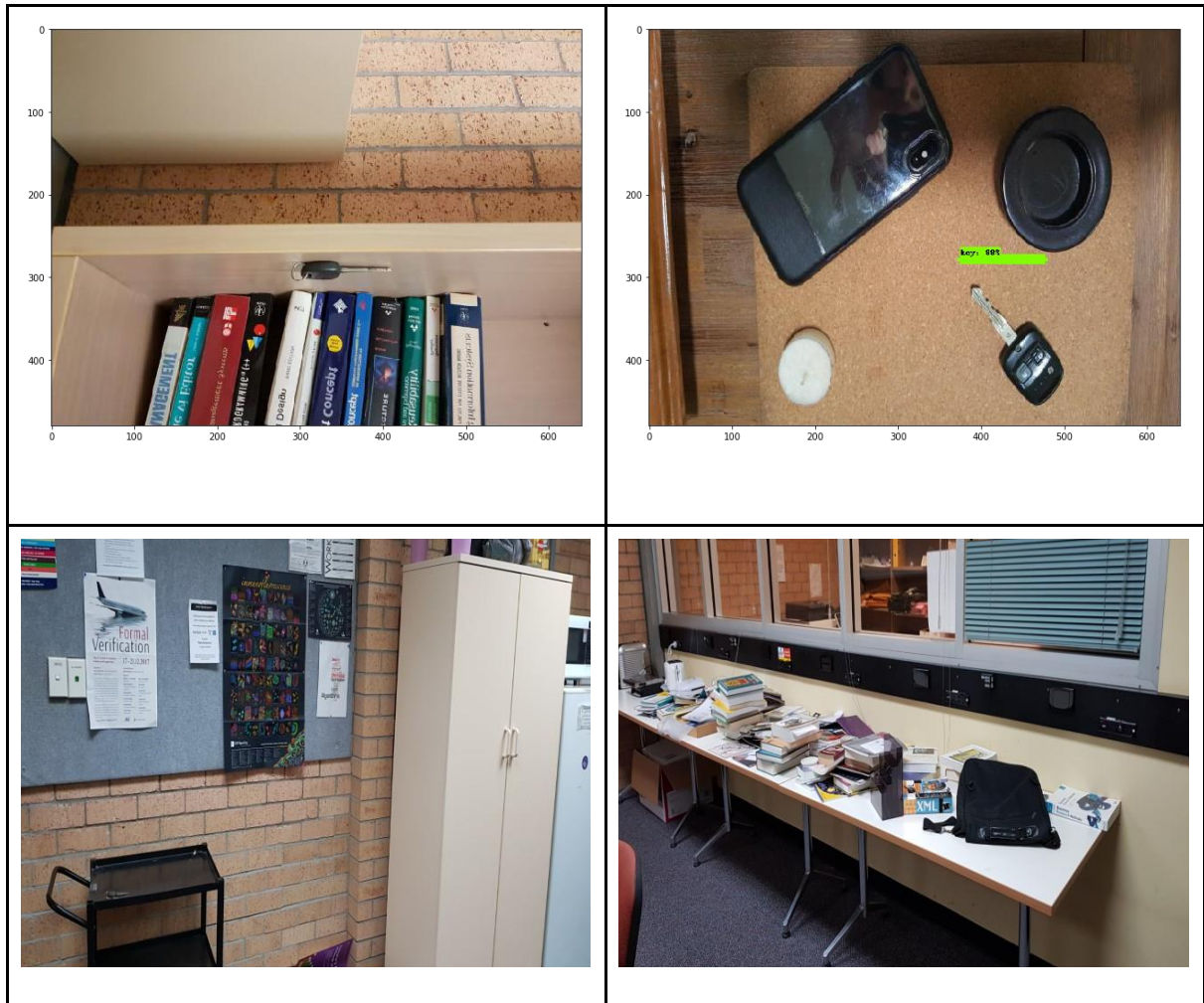


Figure 12: Top-Left- Unclassified key, Top-Right- misclassified key, Bottom: noisy classification problems

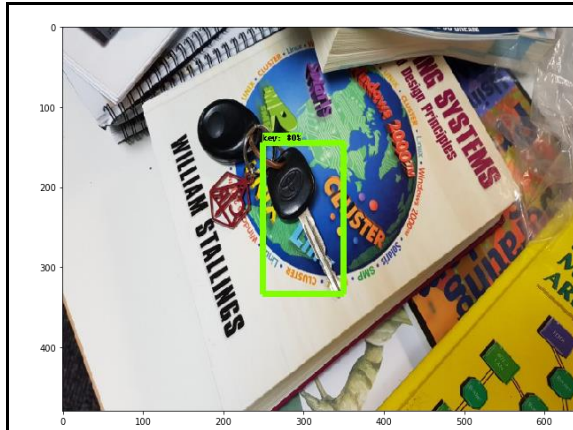
The next step taken to train the network was to cull a large number of the difficult images, and training the network for longer. The image set was repopulated with easier images to bring the total training set back to 650, and then left to train for 6 hours.

The network's accuracy began stagnating. Training was stopped and the network was tested to determine the classification accuracy. This resulted in very similar problems to the first attempt.

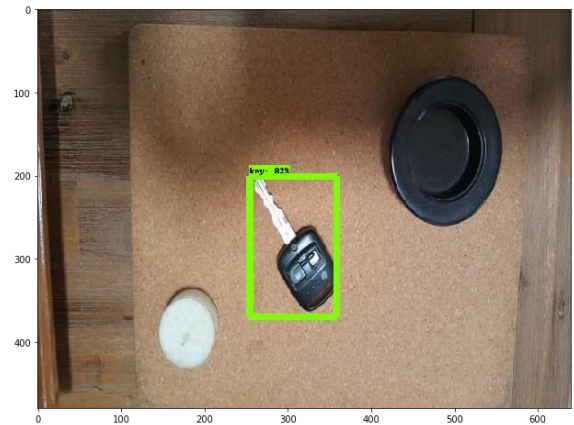
Finally, the network was trained on incrementally more difficult images and trained for a total of 16 hours. This yielded the best results.

Final Results

After running the model for a total of 16 hours training, 420 epochs, a final loss of 1.6 was achieved. The model was run on a testing dataset composed of images not seen before. Out of 50 images, the model successfully drew a bounding box around 27 images, achieving an accuracy of approximately 55%.



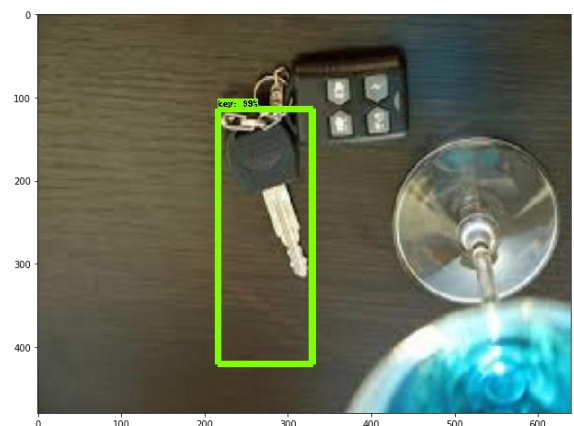
Our model could identify a car key out of a set of keys, even on a background with a lot of noise.



Our model could detect car keys amongst other objects



The labels sometimes correctly located the key however the bounding box was too large



Another loose fitting bounding box

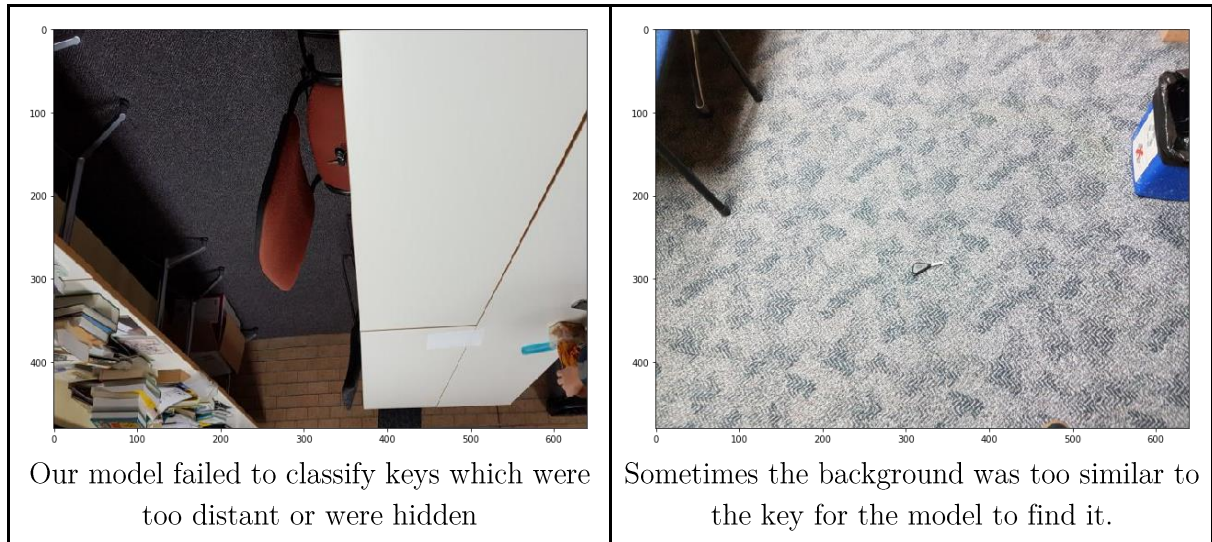


Figure 13 - Classification and localisation of car keys in images using our model

Conclusion

Keys are a complex shape lacking in colour, which may create complications for this model to classify. Many approaches were explored to solve this problem as best as possible, but could not create a concrete solution. The first and second models were entirely insufficient, and our third model couldn't adequately and reliably perform the classifications, however it performed significantly better than our previous attempts with an accuracy of approximately 55%.

For the Markers - Testing Images with our Trained Model

Please ensure you have the following libraries installed:

- TensorFlow (“pip install tensorflow”)
- Jupyter (“pip install jupyter”)
- Unzip the submission and navigate to the “object_detection” folder.
- Copy your testing images into the “test_images” folder. Please guarantee that the images are in the following format “Image (x).jpg” where ‘x’ indicates the image number

Note: We have provided a python script that will rename all ‘.jpg’ files in the “test_images” folder to the appropriate format. This is such that the images can be tested in the Jupyter file.

- Launch Jupyter Notebook from “object-detection”
- Within Jupyter Notebook, open “object_detection_tutorial.ipynb”
- Scroll down in “object_detection_tutorial.ipynb” to the **Detection** subheading.
- You can specify how many images you wish to run at a time:

```
TEST_IMAGE_PATHS = [ os.path.join(PATH_TO_TEST_IMAGES_DIR, 'image  
({}).jpg'.format(i)) for i in range(x, y) ]
```

- Where ‘x’ and ‘y’ are the range of images that you wish to run (For example “for i in range(1,3)” will test the following {“Image (1).jpg, Image (2).jpg, Image (3).jpg”})

Note: There may be a warning that you can only run 20 images at a time due to memory limitations.

- Click on Cell at the top and click “Run All”. This will begin testing the images.
- The results will appear at the bottom of the Jupyter document.

